

**Московский государственный технический
университет им. Н.Э. Баумана**

Факультет информатика и системы управления
Кафедра системы обработки информации и управления

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №1

Выполнил:
студент группы ИУ5-32Б:
Кунев В.
Подпись и дата:

Проверил:
преподаватель каф. ИУ5
Нардид А.Н.
Подпись и дата:

Москва, 2025 г.

Задание:

Разработать программу для решения [биквадратного уравнения](#).

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов A , B , C , вычисляет дискриминант и **ДЕЙСТВИТЕЛЬНЫЕ** корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты A , B , C могут быть заданы в виде параметров командной строки ([вариант задания параметров приведен в конце файла с примером кода](#)). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. [Описание работы с параметрами командной строки](#).
4. Если коэффициент A , B , C введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент — это коэффициент, значение которого, может быть, без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования (кроме C++).

Текст программы

`main.py`

```
"""
Решение биквадратного уравнения
"""

from biquadratic import EquationSolverApp, procedural_main

def main():
    # Спрашиваем у пользователя какую реализацию использовать
    print("Какую реализацию использовать?")
    print("1. ООП")
    print("2. Процедурная")
    choice = input("Выберите 1 или 2: ")
    if choice == "1":
        EquationSolverApp().run()
    elif choice == "2":
        procedural_main()
    else:
        print("Неверный выбор. Попробуйте ещё раз.")

    # print("=== ООП вариант решения биквадратного уравнения ===")
    # oop_main()

    # print("\n=== Процедурный вариант решения биквадратного уравнения ===")
    # procedural_main()

if __name__ == "__main__":
    main()
```

`biquadratic/\_\_init\_\_.py`:

```
"""
Пакет для лабораторной работы №2
Объектно-ориентированные возможности Python
"""

from .oop_biquadratic import EquationSolverApp
from .procedural_biquadratic import main as procedural_main

__all__ = ["EquationSolverApp", "procedural_main"]
```

`biquadratic/procedural\_biquadratic.py`:

```
"""
Решение биквадратного уравнения (процедурный вариант)
"""

from .utils import create_biquadratic_equation_string, parse_args, get_coef,
print_roots, solve_biquadratic

def main():
    print("=== Процедурный вариант решения биквадратного уравнения ===")
    args = parse_args()
    a = args.a if args.a is not None else get_coef("Введите A: ")
    b = args.b if args.b is not None else get_coef("Введите B: ")
    c = args.c if args.c is not None else get_coef("Введите C: ")

    print(f"\nУравнение: {create_biquadratic_equation_string(a, b, c)}")
    roots = solve_biquadratic(a, b, c)
    print_roots(roots)

if __name__ == "__main__":
    main()
```

`biquadratic/ooп\_biquadratic.py`:

```
"""
Решение биквадратного уравнения (ООП вариант)
"""

from .utils import create_biquadratic_equation_string, solve_biquadratic,
get_coef, parse_args, print_roots

class BiquadraticEquation:
    """
    Квадратное уравнение вида  $ax^2 + bx + c = 0$ 
    """

    def __init__(self, a: float, b: float, c: float):
        self.a: float = a
        self.b: float = b
        self.c: float = c

    def solve(self) -> list[float] | list[str]:
        return solve_biquadratic(self.a, self.b, self.c)
```

```

def __str__(self):
    return create_biquadratic_equation_string(self.a, self.b, self.c)

class EquationSolverApp:
    """
    Приложение для решения квадратных уравнений
    """

    def __init__(self):
        self.args = self.parse_args()

    def parse_args(self):
        return parse_args()

    def get_coef(self, prompt: str) -> float:
        return get_coef(prompt)

    def run(self):
        a = self.args.a if self.args.a is not None else self.get_coef("Введите A:")
        b = self.args.b if self.args.b is not None else self.get_coef("Введите B:")
        c = self.args.c if self.args.c is not None else self.get_coef("Введите C:")

        eq = BiquadraticEquation(a, b, c)
        print(f"\nУравнение: {eq}")
        roots = eq.solve()

        print_roots(roots)

if __name__ == "__main__":
    EquationSolverApp().run()

```

`biquadratic/untils.py`:

```

"""
Вспомогательные функции
"""

import argparse
import math

INFINITY_ROOTS_TEXT = "Бесконечное множество решений"

```

```

def create_biquadratic_equation_string(a: float, b: float, c: float) -> str:
    """
    Создание строки для квадратного уравнения

    Args:
        a (float): Коэффициент A
        b (float): Коэффициент B
        c (float): Коэффициент C

    Returns:
        str: Строка для квадратного уравнения
    """

    def int_or_float(x: float) -> int | float:
        return int(x) if x.is_integer() else x

    def get_sign(x: float) -> str:
        return "+" if x >= 0 else "-"

    return f"{int_or_float(a)}x^4 {get_sign(b)} {abs(int_or_float(b))}x^2 {get_sign(c)} {abs(int_or_float(c))} = 0"

def parse_args() -> argparse.Namespace:
    """
    Парсинг аргументов командной строки

    Returns:
        argparse.Namespace: Объект с аргументами
    """
    parser = argparse.ArgumentParser(
        description="Решение биквадратного уравнения (процедурный вариант)"
    )
    parser.add_argument("a", nargs="?", type=float, help="Коэффициент A")
    parser.add_argument("b", nargs="?", type=float, help="Коэффициент B")
    parser.add_argument("c", nargs="?", type=float, help="Коэффициент C")
    return parser.parse_args()

def get_coef(prompt: str) -> float:
    """
    Получение коэффициента

    Args:
        prompt (str): Просим пользователя ввести коэффициент

    Returns:
        float: Коэффициент
    """
    while True:
        try:

```

```

        return float(input(prompt))
    except ValueError:
        print("Некорректный ввод. Попробуйте снова.")

def print_roots(roots: list[float] | list[str]):
    """
    Вывод корней

    Args:
        roots (list[float] | list[str]): Список корней или текст об бесконечном
        количестве корней
    """
    if not roots:
        print("Действительных корней нет")
    elif roots == [INFINITY_ROOTS_TEXT]:
        print(INFINITY_ROOTS_TEXT)
    else:
        print(f"Найдено {len(roots)} действительных корней:")
        for i, r in enumerate(roots, 1):
            print(f"  x{i} = {r:.4f}")

def solve_biquadratic(a: float, b: float, c: float) -> list[float] | list[str]:
    """
    Решение биквадратного уравнения

    Args:
        a (float): Коэффициент A
        b (float): Коэффициент B
        c (float): Коэффициент C

    Returns:
        list[float] | list[str]: Список корней или текст об бесконечном
        количестве корней
    """
    if a == 0 and b == 0 and c == 0:
        return [INFINITY_ROOTS_TEXT]

    if a == 0:
        if b == 0:
            return []
        y = -c / b
        if y < 0:
            return []
        return sorted({math.sqrt(y), -math.sqrt(y)})

    d = b * b - 4 * a * c
    if d < 0:
        return []

```

```

roots = []
for y in [(-b + math.sqrt(d)) / (2 * a), (-b - math.sqrt(d)) / (2 * a)]:
    if y >= 0:
        roots.extend([math.sqrt(y), -math.sqrt(y)])
return sorted(set(roots))

```

`BiquadraticEquationSolver.java`:

```

import java.util.*;

public class BiquadraticEquationSolver {

    private static final String INFINITY_ROOTS_TEXT = "Бесконечное множество
решений";
    private static final Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {
        System.out.println("=== Процедурный вариант решения биквадратного
уравнения ===");

        scanner.useLocale(Locale.US);

        double a, b, c;

        if (args.length >= 3) {
            a = Double.parseDouble(args[0]);
            b = Double.parseDouble(args[1]);
            c = Double.parseDouble(args[2]);
        } else {
            a = getCoefficient("Введите A: ");
            b = getCoefficient("Введите B: ");
            c = getCoefficient("Введите C: ");
        }

        System.out.println("\nУравнение: " + createBiquadraticEquationString(a,
b, c));
        List<Object> roots = solveBiquadratic(a, b, c);
        printRoots(roots);
    }

    /**
     * Создание строки для квадратного уравнения
     */
    public static String createBiquadraticEquationString(double a, double b,
double c) {
        String aStr = formatNumber(a);
        String bStr = formatNumber(Math.abs(b));
        String cStr = formatNumber(Math.abs(c));
    }

```



```

        String bSign = b >= 0 ? "+" : "-";
        String cSign = c >= 0 ? "+" : "-";

        return aStr + "x^4 " + bSign + " " + bStr + "x^2 " + cSign + " " + cStr +
" = 0";
    }

    /**
     * Форматирование числа (целое или дробное)
     */
    private static String formatNumber(double x) {
        if (x == (int) x) {
            return Integer.toString((int) x);
        }
        return Double.toString(x);
    }

    /**
     * Получение коэффициента с клавиатуры
     */
    public static double getCoefficient(String prompt) {
        while (true) {
            try {
                System.out.print(prompt);
                return scanner.nextDouble();
            } catch (InputMismatchException e) {
                System.out.println("Некорректный ввод. Попробуйте снова.");
                scanner.next(); // очистить буфер
            }
        }
    }

    /**
     * Вывод корней
     */
    public static void printRoots(List<Object> roots) {
        if (roots.isEmpty()) {
            System.out.println("Действительных корней нет");
        } else if (roots.size() == 1 && roots.get(0) instanceof String) {
            System.out.println(roots.get(0));
        } else {
            System.out.println("Найдено " + roots.size() + " действительных
корней:");
            for (int i = 0; i < roots.size(); i++) {
                System.out.printf("  x%d = %.4f\n", i + 1, (Double)
roots.get(i));
            }
        }
    }

    /**

```

```

* Решение биквадратного уравнения
*/
public static List<Object> solveBiquadratic(double a, double b, double c) {
    if (a == 0 && b == 0 && c == 0) {
        return Arrays.asList(INFINITY_ROOTS_TEXT);
    }

    if (a == 0) {
        if (b == 0) {
            return new ArrayList<>();
        }
        double y = -c / b;
        if (y < 0) {
            return new ArrayList<>();
        }
        List<Object> roots = new ArrayList<>();
        roots.add(Math.sqrt(y));
        roots.add(-Math.sqrt(y));
        return sortRoots(roots);
    }

    double discriminant = b * b - 4 * a * c;
    if (discriminant < 0) {
        return new ArrayList<>();
    }

    List<Object> roots = new ArrayList<>();
    double y1 = (-b + Math.sqrt(discriminant)) / (2 * a);
    double y2 = (-b - Math.sqrt(discriminant)) / (2 * a);

    if (y1 >= 0) {
        roots.add(Math.sqrt(y1));
        roots.add(-Math.sqrt(y1));
    }

    if (y2 >= 0) {
        roots.add(Math.sqrt(y2));
        roots.add(-Math.sqrt(y2));
    }

    return sortRoots(removeDuplicates(roots));
}

/**
* Удаление дубликатов корней
*/
private static List<Object> removeDuplicates(List<Object> roots) {
    Set<Object> uniqueRoots = new LinkedHashSet<>();
    for (Object root : roots) {
        if (root instanceof Double) {

```

```

        // Округляем для сравнения (из-за погрешности вычислений с
плавающей точкой)
        double rounded = Math.round((Double) root * 1e10) / 1e10;
        uniqueRoots.add(rounded);
    } else {
        uniqueRoots.add(root);
    }
}
return new ArrayList<>(uniqueRoots);
}

/**
 * Сортировка корней
 */
private static List<Object> sortRoots(List<Object> roots) {
    List<Double> numericRoots = new ArrayList<>();
    List<Object> otherRoots = new ArrayList<>();

    for (Object root : roots) {
        if (root instanceof Double) {
            numericRoots.add((Double) root);
        } else {
            otherRoots.add(root);
        }
    }

    Collections.sort(numericRoots);
    List<Object> result = new ArrayList<>();
    result.addAll(otherRoots);
    result.addAll(numericRoots);

    return result;
}
}

```

Скриншот работы приложения

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab1$ python ./main.py
Какую реализацию использовать?
1. ООП
2. Процедурная
Выберите 1 или 2: 1
Введите A: 6
Введите B: 3
Введите C: -4

Уравнение:  $6x^4 + 3x^2 - 4 = 0$ 
Найдено 2 действительных корней:
x1 = -0.7771
x2 = 0.7771
```

Рисунок 1 ООП подход

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab1$ python ./main.py
Какую реализацию использовать?
1. ООП
2. Процедурная
Выберите 1 или 2: 2
=== Процедурный вариант решения биквадратного уравнения ===
Введите A: 0
Введите B: 0
Введите C: 0

Уравнение:  $0x^4 + 0x^2 + 0 = 0$ 
Бесконечное множество решений
```

Рисунок 2 Процедурный подход

```
(.venv) cunev@ROGStrixG814JIR:~/VSCode/ParadigmsAndConstructsOfProgrammingLanguages/lab1$ java BiquadraticEquationSolver
=== Процедурный вариант решения биквадратного уравнения ===
Введите A: 0
Введите B: 0
Введите C: -1

Уравнение:  $0x^4 + 0x^2 - 1 = 0$ 
Действительных корней нет
```

Рисунок 3 Реализация на Java